

ME315_Lab3

pacotes

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

filter, lag

Os seguintes objetos são mascarados por 'package:base':

intersect, setdiff, setequal, union

```
library(stringr)
library(leaflet)
```

1) Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
- Para o arquivo de vôos:
 1. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 2. Leia apenas 1 milhão de linhas por vez;
 3. Remova vôos em que o aeroporto de destino comece com a letra 1;
 4. Remova registros em que existam pelo menos uma coluna faltante;
 5. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 6. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
#apenas importar
airlines <- read_csv("//smb/ra183541/Downloads/archive (1)/airlines.csv")
```

Rows: 14 Columns: 2

— Column specification —

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

- Use ``spec()`` to retrieve the full column specification for this data.
- Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
#importar columnas específicas
airports <- read_csv("//smb/ra183541/Downloads/archive (1)/airports.csv",
                     col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

Rows: 322 Columns: 5

— Column specification —

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

- Use ``spec()`` to retrieve the full column specification for this data.
- Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
#flights
acumular_estadisticas <- function(df, pos) {
  df %>%
    select(DESTINATION_AIRPORT, ARRIVAL_DELAY) %>%
    filter(!str_detect(DESTINATION_AIRPORT, "^1")) %>%
    na.omit() %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY),
      n_voos = n(),
      .groups = "drop"
    )
}

resultados_chunks <- read_csv_chunked("//smb/ra183541/Downloads/archive (1)/flights.csv",
                                       callback = DataFrameCallback$new(acumular_estadisticas),
                                       chunk_size = 1000000
)
```

— Column specification —

```
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
```

- Use ``spec()`` for the full column specifications.

```
flights <- resultados_chunks %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_total = sum(soma_atraso),
    n_total = sum(n_voos),
    media_atraso = soma_total / n_total,
    .groups = "drop"
  )
```

2) Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

1. Identifique a coluna que é a chave na tabela `flights`;
2. Identifique a coluna que é a chave na tabela `airports`;
3. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
4. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
5. Armazene a tabela resultante no objeto `flights`.

```
aeroportos_ausentes <- setdiff(flights$DESTINATION_AIRPORT, airports$IATA_CODE)
print("ausentes em airports:")
```

```
[1] "ausentes em airports:"
```

```
print(aeroportos_ausentes)
```

```
character(0)
```

```
flights <- flights %>%
  left_join(
    airports %>% select(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
```

3) Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aeroportos_por_estado <- airports %>%
  filter(!is.na(STATE)) %>%
  group_by(STATE) %>%
  summarise(qtd_aeroportos = n()) %>%
  arrange(desc(qtd_aeroportos))

print(aeroportos_por_estado)
```

```
# A tibble: 54 × 2
  STATE qtd_aeroportos
  <chr>         <int>
```

1 TX	24
2 CA	22
3 AK	19
4 FL	17
5 MI	15
6 NY	14
7 CO	10
8 MN	8
9 MT	8
10 NC	8

i 44 more rows

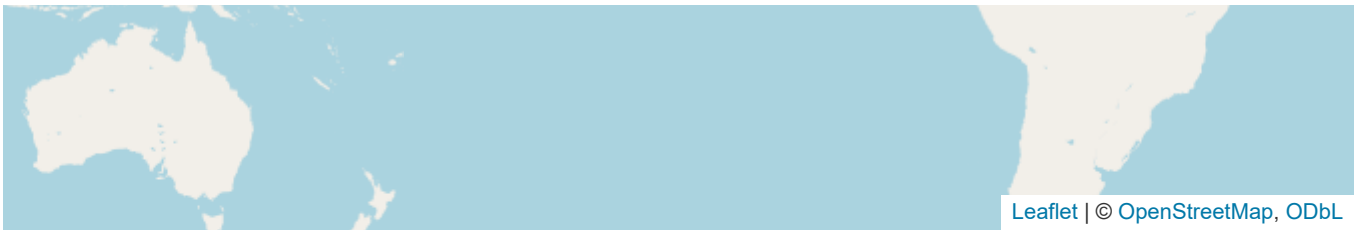
4) Apresente um mapa representando todos os atrasos observados por aeroporto.

1. Carregue o pacote `leaflet`;
2. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
3. Armazene o grafico em b) numa variável chamada `this`;
4. Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```
dados_mapa <- flights %>%
  filter(!is.na(LATITUDE) & !is.na(LONGITUDE))

this <- leaflet(dados_mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0("<b>Aeroporto:</b> ", DESTINATION_AIRPORT, "<br>",
                    "<b>Atraso Médio:</b> ", round(media_atraso, 2), " min")
  )
this
```





Data/hora

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 20:30:41 -03"
```